

DATE	30 September 2026
TEAM ID	SWUID2026349965
PROJECT NAME	PocketSmart AI: Your Smart Budget & Recommendation Assistant

Phase 05: Project Development

1. Introduction

This document describes how PocketSmart AI was developed, following the five milestones of the project brief. The application is built with FastAPI and Google Gemini 2.5, with a Jinja2, HTML and CSS frontend.

1.1 Project Folder Structure

Item	Purpose
Main application file	FastAPI application, routes, authentication, sessions and recommendation functions
Environment file (.env)	Google API key and secret key for tokens
static/uploads	Uploaded outfit images and the stylesheet
templates	dashboard, history, home planner, index, jewelry planner, login, party planner and register pages
Virtual environment	Isolated Python packages for the project

2. Milestone 1: Gemini AI Initialization

The objective of this stage was to establish authenticated access to Gemini 2.5 for both text reasoning and image analysis (used by the Jewelry planner).

Activity	Description
1.1 Google account and API	Sign in to Google AI Studio, accept the terms and choose Get API key. The model can be selected in the model list or set through the API documentation.
1.2 Access and permissions	Open the Dashboard, choose API Keys, click Create API key and copy the generated key. The key is stored in the environment file and never written in the source code.
1.3 Validate connectivity	A short Python test sent a text prompt and an image-with-text prompt to confirm that both text-only and multimodal requests work before building the application.

3. Milestone 2: Core Functionalities

3.1 Application Setup

- Import the required libraries: FastAPI, Uvicorn, Jinja2, Pillow, Pydantic, passlib, python-jose, python-dotenv and the Google Generative AI SDK.
- Create the FastAPI application titled PocketSmart: AI Budget Planner.
- Load the environment file, read the Google API key (with GEMINI_API_KEY as an alternative name) and stop with a clear error if no key is found.
- Configure password hashing (bcrypt), token settings (HS256 algorithm, 30-minute expiry) and the OAuth2 bearer scheme with optional token.
- Enable CORS, set up the Jinja2 templates folder, create the upload folder and mount the static files.

3.2 Recommendation Functions

Function	Behaviour
Home recommendations	Builds a prompt with the budget in INR, counts of lights, fans, furniture and dining tables, the selected rooms and extra notes. Asks Gemini for a JSON breakdown, extracts the JSON from the reply and adds shopping links (Amazon, Flipkart, IKEA, Myntra, Ajio) using each item's search terms.
Party recommendations	Builds a prompt with party type, guests, venue and needs. After the reply, the backend builds an INR calculation table (items, total cost, percentage of budget per category) and attaches only the platforms relevant to each category, such as Swiggy and Zomato for catering or OYO and MakeMyTrip for venues. Venue suggestions receive their own search links.
Jewelry recommendations	Builds a prompt with budget, occasion and preferences. If an outfit image is provided, it is opened with Pillow and sent together with the text so Gemini can analyse colours, style and formality. Links are added for Amazon, Flipkart, Bluestone, Tanishq, CaratLane, Melorra and Meesho.
Currency helper	Converts USD to INR with a configurable exchange rate.

3.3 Authentication Pages and Routes

Route	Function
Login page	Shows the login form; users who are already signed in are redirected to the dashboard.
Register page	Shows the registration form with the same redirect rule.
Logout	Adds the token to a blacklist, removes the user's active session, deletes the cookie and redirects to the login page.
Token	Verifies the username and password, creates the access token, replaces any older session token and stores the token in an HTTP-only cookie.
Session information	Returns username, login time, last activity, session duration in minutes and stored session data.
Session data	Updates the stored session data and last activity time.

4. Milestone 3: Backend Integration

Endpoint	Development notes
Home budget	Saves the latest home budget (time, budget, requested counts) in the session, calls the home recommendation function and saves the result to history.
Party budget	Saves party type, guests and budget in the session, calls the party recommendation function and saves the result to history.
Jewelry budget	Receives form fields and an optional file upload, saves the image to the upload folder, records whether an image was used, calls the jewelry function and saves the result with the image name.
Recommendation history	Returns the user's saved plans sorted newest first with id, time, type, input summary and result summary.
Recommendation details	Returns the complete stored result of one plan by its id, or a not-found error.
History page	Serves the history template for the logged-in user.
Startup task	A background task runs every 5 minutes and removes sessions inactive for more than 30 minutes.
Entry point	When the file is run directly, Uvicorn starts the application on host 0.0.0.0 and port 8000.

4.1 Error Handling

Each recommendation function is wrapped so that any failure returns an HTTP 500 response with a clear message. Protected routes depend on the active-user check, so requests without a valid token are rejected.

5. Milestone 4: User Interface

Page	Description
Landing page	Introduces PocketSmart with a hero section, the three planner cards, testimonials, a call-to-action and a footer with company, product, resource and legal links.
Register	Fields for username, email, password and confirm password with a link to sign in.
Login	Username and password fields, forgot-password link and a link to create an account.
Dashboard	Welcome message, cards for Home, Party and Jewelry planners, a button to view all history and a Recent Activity list.
Home planner form	Total budget, number of lights, ceiling fans, furniture pieces and dining tables, room checkboxes and a notes box.
Home results	Budget summary with total and remaining budget, category sections with allocation, item tables with description, price, quantity and shopping link buttons, and additional suggestions.
Party planner form	Budget, guests, party type, venue type, need selectors for catering, decoration and entertainment, and a notes box.
Party results	Category-wise cost list with Shop-on links, total, allocated and remaining amounts, venue suggestions and additional suggestions, with a Print/Save option.
Jewelry planner form	Budget, occasion, style preferences and an outfit image upload with preview and remove option.
Jewelry results	Budget summary, outfit analysis (colours, style, formality), jewelry cards with description, style, price and shop links, and styling tips.
History	Cards showing each past plan with type, date, total and remaining budget, key inputs and a View Full Details button.

6. Milestone 5: Testing and Optimization

- End-to-end testing with varied budgets across all three planners.
- Evaluation of Gemini response quality, budget adherence and platform accuracy.
- Prompt refinement and validation to prevent input edge-case failures.
- Fallback recommendations when the AI returns insufficient results.
- Improved session handling and a smooth interface across devices.

6.1 Conclusion

The development phase produced a modular FastAPI application that connects a Jinja2 frontend to Gemini 2.5 and delivers budget-aware recommendations for home, party and jewelry planning. Detailed test results are recorded in Phase 06.